



KAZAKH-BRITISH
TECHNICAL
UNIVERSITY

Programming Principles I

Lecture 2



Zhiger Bolatov

Senior Lecturer
School of IT&I

- Relational Operators
- Logical Operators
- Conditional statements
- Math Functions
- String
- Character arrays
- Loops (for, while, do-while)
- Continue
- Break

Relational Operators

- Relational operators are used to form conditions in an expression.
- They are mainly used to compare variables with other variables or constants.
- They are binary in nature

Operator	Description	Expression
>	Greater than	$a > b$
<	Less than	$a < b$
>=	Greater than or equal to	$a \geq b$
<=	Less than or equal to	$a \leq b$
==	Equal to	$a == b$
!=	Not equal to	$a != b$

Relational Operators

The result of a relational expression is either true (1) or false (0).

Variable value	Example Expression	Result
a=9, b=8	a>b	True or 1
	a>=b	True or 1
	a==b	False or 0
	a!=b	True or 1
	a>2	True or 1
a=9, b=9	a>=b	True or 1
	a==b	True or 1
	a!=b	False or 0
	b>9	False or 0

Relational Operators

Using Relational Operators

```
#include <iostream>
using namespace std;
int main() {
    int a=2, b=1, c=4, d=6, e=9, f=-6;
    cout << (a>b) << endl;    // True(1)
    cout << (b>c) << endl;    // False(0)
    cout << (d<=e) << endl;   // True(1)
    cout << (e!=f) << endl;   // True(1)
    cout << (c>=a) << endl;   // True(1)
    cout << (a==d);           // False(0)
    return 0;
}
```

Logical Operators

- These operators are used to combine two or more conditions.
- They return true or false.
- In the table x and y denote conditions

Operator		Type	Example
&&	Logical And	Binary	x && y
	Logical Or	Binary	x y
!	Logical Not	Unary	!x

Logical Operators

Working of AND (&&) operator

Variable value	Expression	Condition 1	Result	Condition 2	Result	Output of expression
x=15, y=7	x>y && y>6	x>y	True(1)	y>6	True(1)	True(1)
	x>y && y>6	x>y	True(1)	y>6	True(1)	True(1)
	x>y && y>6	x>y	True(1)	y>6	True(1)	True(1)
	x>y && y>6	x>y	True(1)	y>6	True(1)	True(1)

Logical Operators

Working of OR (||) operator

Variable value	Expression	Condition 1	Result	Condition 2	Result	Output of expression
x=15, y=7	x>y y >6	x>y	True(1)	y>6	True(1)	True(1)
	x>y y!= 7	x>y	True(1)	y!=7	False(0)	True(1)
	x<y y==7	x<y	False(0)	y==7	True(1)	True(1)
	x<y y>7	x<y	False(0)	y>7	False(0)	False(0)

Logical Operators

Working of NOT (!) operator

Variable value	Expression	Condition	Result	!(Condition)	Output of expression
x=15, y=4	! (x>y)	x>y	True(1)	! (True)	False(0)
	! (y>7)	y>7	False(0)	! (False)	True(1)
	!(x>y y!= 4)	T F	True(1)	!(True)	False(0)
	!(x<y y>7)	F F	False(0)	!(False)	True(1)

Conditional statements

Conditional statements allow a program to make decisions and execute different code depending on whether a condition is true or false. For example: `int score = 30;`

Main types in C++:

- if — executes code when a condition is true
- if...else — chooses between two alternatives
- else if — checks multiple conditions

Syntax: `if (condition1) {
 // statements
} else if (condition2) {
 // statements
} else {
 // statements
}`

```
if (score >= 50) {  
    cout << "Pass";  
} else {  
    cout << "Fail";  
}
```

Conditional statements

The program checks a student's score and whether they received extra credit.

First, the if statement checks whether the score is 90 or higher AND extra credit is received. Both conditions must be true to print "Excellent!".

If this condition is false, the else if statement checks whether the score is 70 or higher OR extra credit is received. Only one of these conditions needs to be true to print "Good!".

If neither condition is true, the else statement prints "Needs improvement."

For score = 75 and extraCredit = true:

The first condition is false, but the second condition is true, so the program outputs "Good!".

```
int score = 75;
bool extraCredit = true;

if (score >= 90 && extraCredit) {
    cout << "Excellent!";
}
else if (score >= 70 || extraCredit) {
    cout << "Good!";
}
else {
    cout << "Needs improvement.";
}
```

Math Functions

C++ provides useful mathematical functions through the `<cmath>` library.

Function	Description	Example	Result
<code>pow(x, y)</code>	Raises x to the power of y	<code>pow(2, 3)</code>	8
<code>round(x)</code>	Rounds to the nearest integer	<code>round(4.6)</code>	5
<code>ceil(x)</code>	Rounds up to the nearest integer	<code>ceil(4.2)</code>	5
<code>floor(x)</code>	Rounds down to the nearest integer	<code>floor(4.8)</code>	4

```
#include <cmath>
cout << pow(2, 3);    // 8
cout << round(4.6);  // 5
cout << ceil(4.2);   // 5
cout << floor(4.8);  // 4
```

4.2 → round → 4
4.2 → ceil → 5
4.2 → floor → 4

4.8 → round → 5
4.8 → ceil → 5
4.8 → floor → 4

A string is a collection of characters. It is usually a meaningful sequence representing the name of an entity.

Generally, it is combination of two or more characters enclosed in double quotes.

"Good Morning"	// string with 2 words
"Mehak"	// string with one word
"B.P."	// string with letters and symbols
""	// an empty string

The above examples are also known as string literals

How to store and process strings in C++?

- C++ does not provide with a special data type to store strings.
- Thus we use arrays of the type `char` to store strings
- Strings in C++ are always terminated using a null character represented by `'\0'`
- Thus strings are basically one dimensional character arrays terminated by a null character (`'\0'`)
- String literals are the values stored in the character array

Character arrays

Character arrays: declaration

To declare a character array or string the syntax is :

```
char arrayname[size];
```

Example:

```
char name[20];
```

Here name is a character array or string capable of storing maximum of **19 characters**. One character is reserved for storing **'\0'**.

Thus the number of elements that can be stored in a string is always $n-1$, if the size of the array specified is n . This is because 1 byte is reserved for the NULL character **'\0'** i.e. backslash zero.

Examples of array declarations

Array declaration	Used to store
<code>char city[20];</code>	name of a city
<code>char address[40];</code>	address of a person
<code>char designation[15];</code>	designation
<code>char author[25];</code>	name of author

Character arrays

Character arrays : Initialization

```
char name[20]="Jane Eyre";
```

string

null character



name[0]

name[1]

name[7]

name[9]

The above string will have 9 characters and 1 space for the null.

Thus size of name will be 10.

Character arrays

Character arrays: Initialization

We can also initialize a character array in this way:

```
char name[ ]="Jane Eyre";
```

- In this case the array is initialized to the mentioned string and the size of the array is the number of characters in the string literal. Here it is 10.
- In the above declarations the null character is automatically inserted at the end of the string.

We can also initialize a string in this way:

```
char name[ ]={'J', 'a', 'n', 'e', ' ', 'E', 'y', 'r', 'e', '\0'};
```

Note :

- Here the character array or string is initialized character by character.
- The '\0' has to be inserted at the end. This is because in this method the null character has to be inserted by the programmer.

Character arrays

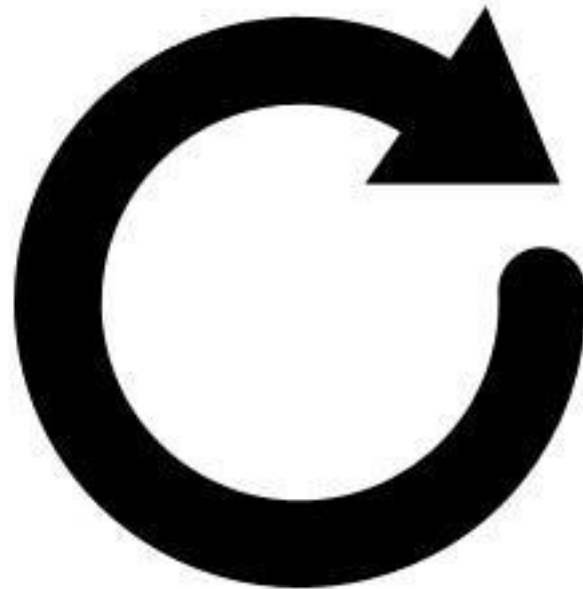
Some more examples of string initialization

Initialization	Memory representation
<code>char animal[]="Lion";</code>	L i o n '\0'
<code>char location[]="New Delhi";</code>	N e w D e l h i '\0'
<code>char serial_no[]="A011";</code>	A 0 1 1 '\0'
<code>char company[]="Airtel";</code>	A i r t e l '\0'

Loops

Loops are basically used when there is a need to perform some operation more than once or n number of times.

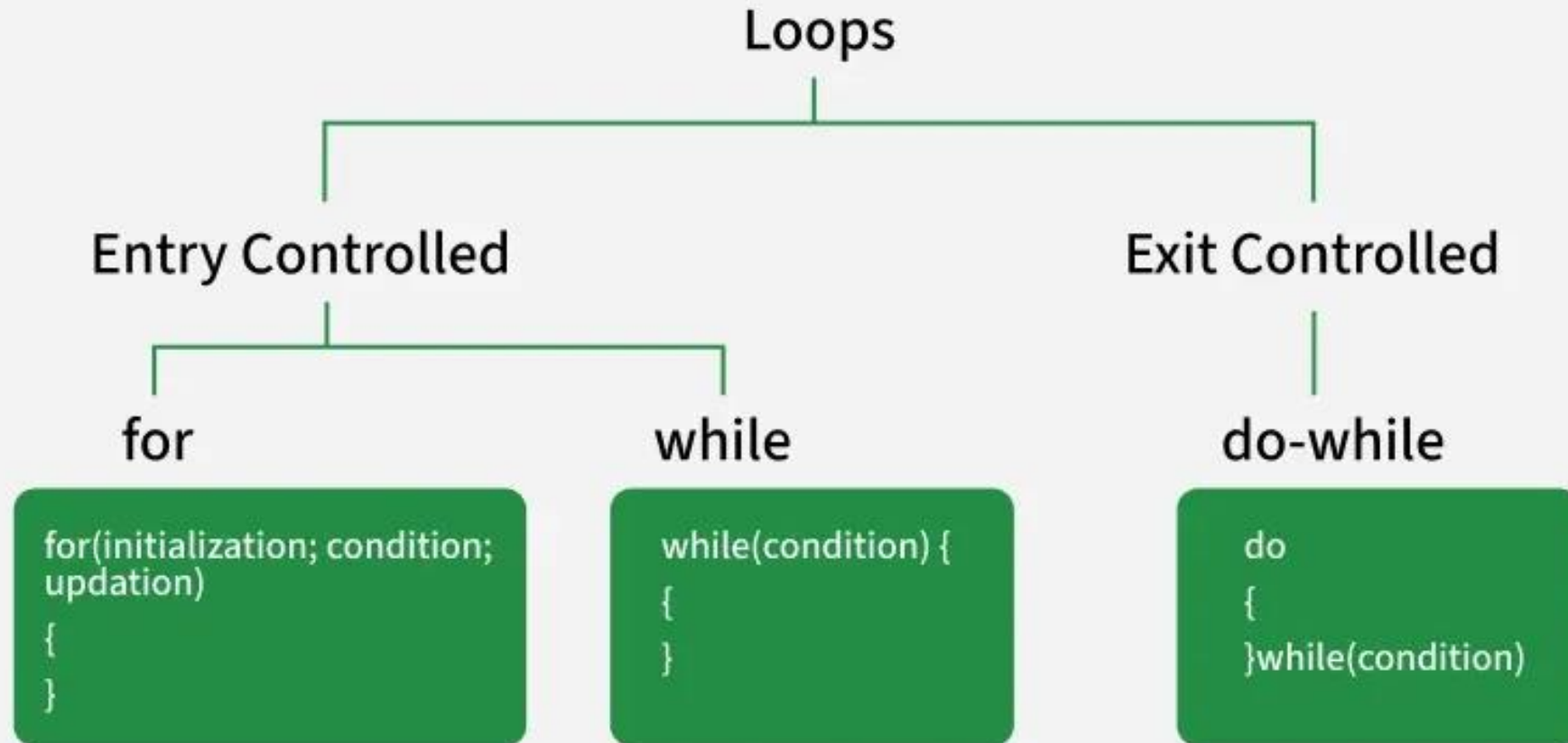
Loops come into use when we need to repeatedly execute a block of statements.



LOOPS REPEAT
ACTIONS...

SO YOU DON'T HAVE TO

Type of Loops



for and **while** are **entry-controlled loops**, meaning the condition is checked before the loop body executes, while **do-while** is an **exit-controlled loop**, so the body executes at least once before the condition is checked.

FOR Loop

A **for** loop is used to repeat a block of code a specific number of times or while a condition remains true. It is an entry-controlled loop, so the condition is checked before each iteration.

Syntax:

```
for (initialization; condition; update) {  
    // statements  
}
```

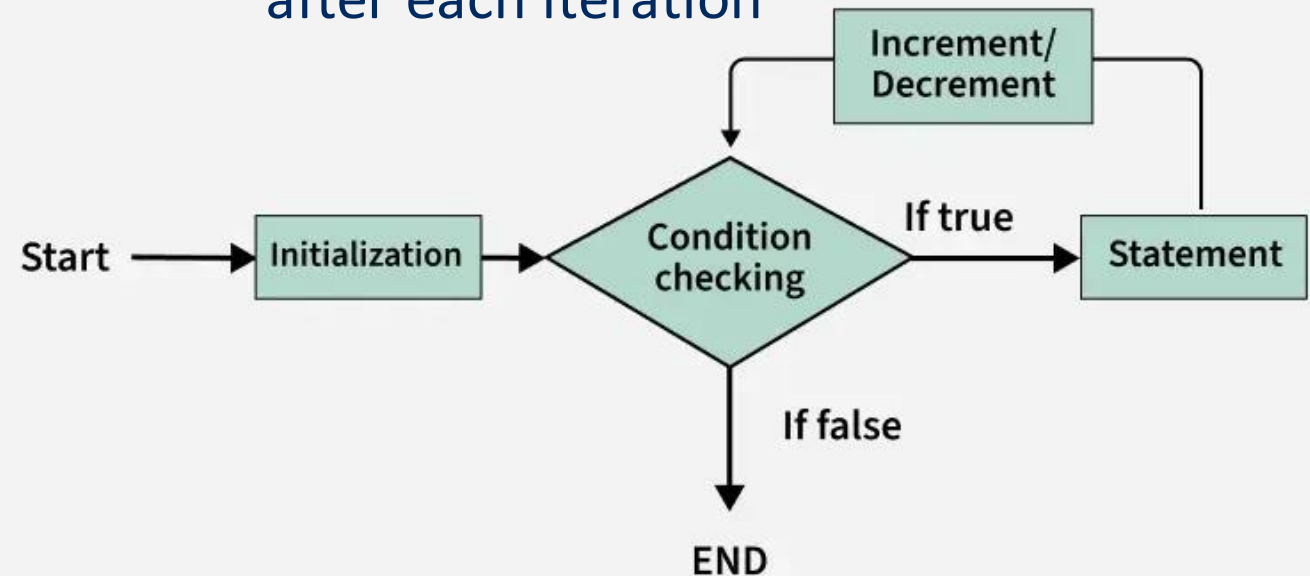
Example:

```
for (int i = 1; i <= 5; i++) {  
    cout << i << " ";  
}
```

Result:

1 2 3 4 5

- Initialization — sets the starting value
- Condition — determines whether the loop continues
- Update — changes the loop variable after each iteration



FOR Loop

Example: Calculate the Sum of Numbers. Calculate the sum of all integers from 1 to n.

Code:

```
int n;  
cin >> n;  
int sum = 0;  
  
for (int i = 1; i <= n; i++) {  
    sum = sum + i;  
}  
cout << "Sum = " << sum;
```

Result: For n = 5

Sum = 15

WHILE Loop

A **while** loop is also an entry-controlled loop in which the condition is checked before entering the body.

Syntax:

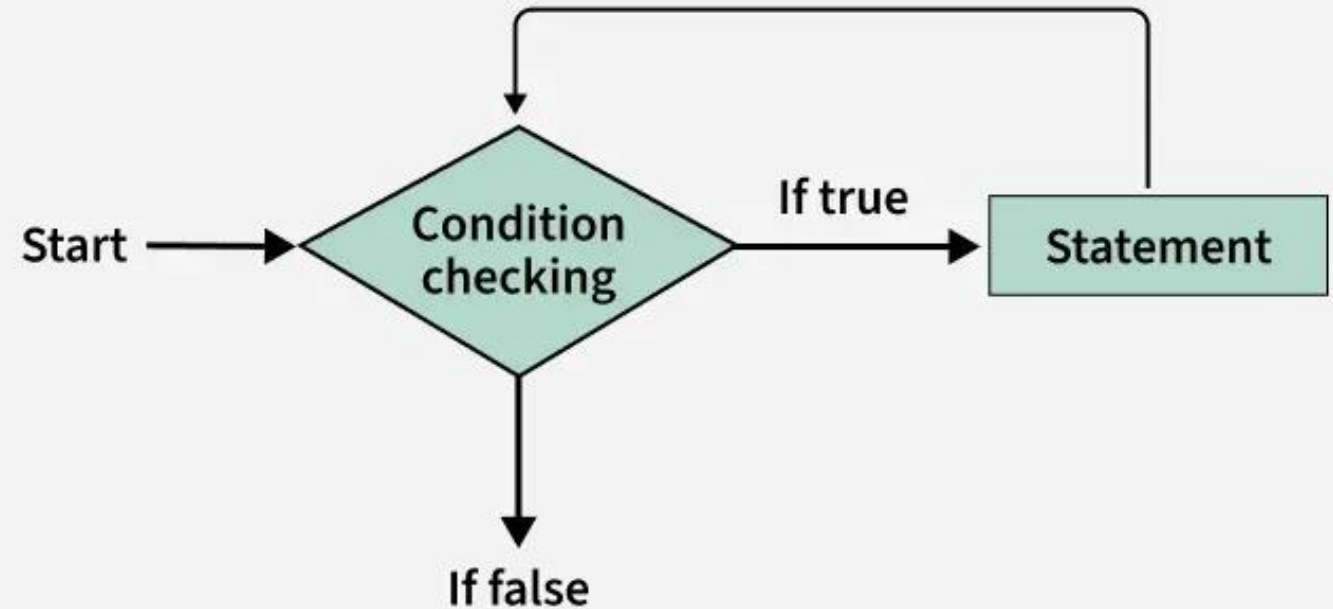
```
while (condition) {  
    // statements  
}
```

Example:

```
int i = 1;  
while (i <= 5) {  
    cout << i << " ";  
    i++;  
}
```

Result:

1 2 3 4 5



WHILE Loop

Example: Password Check. Keep asking the user for input password until they enter a valid value.

Code:

```
string password;  
cout << "Enter password: ";  
cin >> password;  
  
while (password != "1234") {  
    cout << "Wrong password. Try again: ";  
    cin >> password;  
}  
  
cout << "Access granted!";
```

DO-WHILE Loop

The **do-while** loop is an exit-controlled loop, which means that the condition is checked after executing the loop body. Due to this, the loop body will execute at least once irrespective of the test condition.

Syntax:

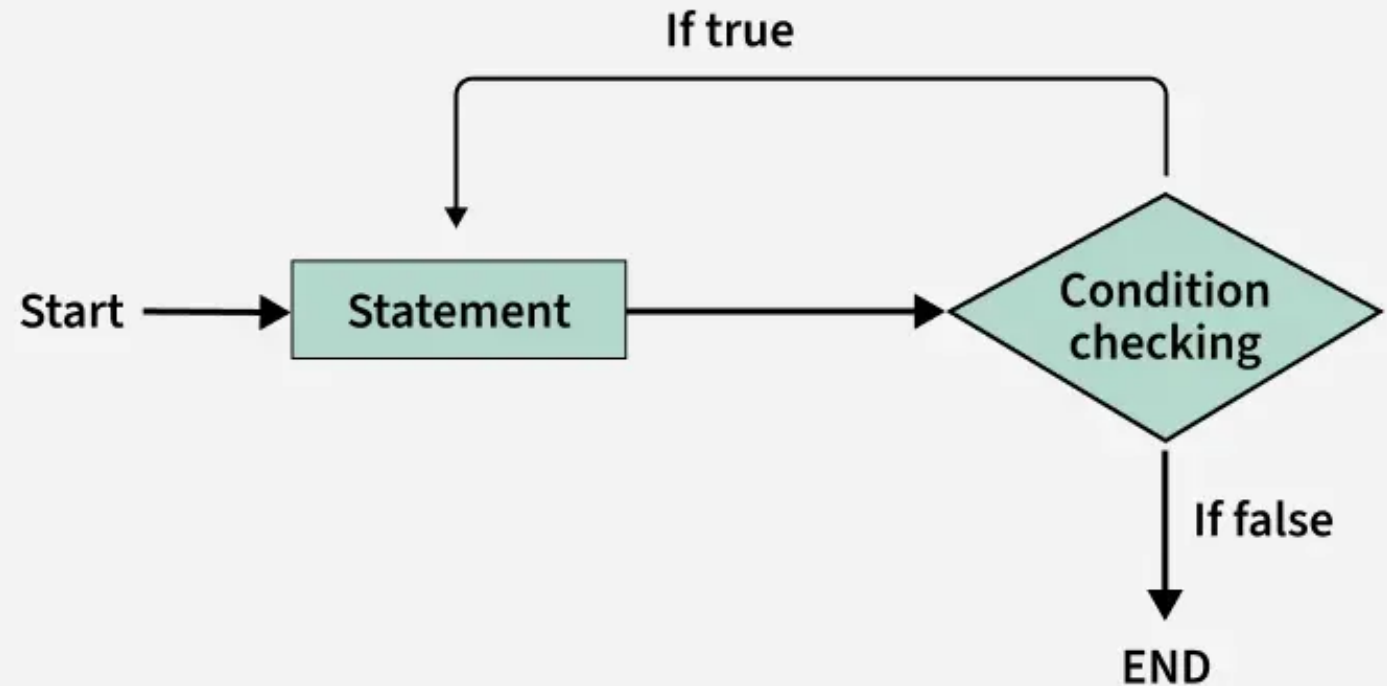
```
do {  
    // statements  
} while (condition)
```

Example:

```
int i = 1;  
do {  
    cout << i << " ";  
    i++;  
} while (i <= 5)
```

Result:

1 2 3 4 5



DO-WHILE Loop

Example: Menu that should be displayed at least once.

Code:

```
int choice;
do {
    cout << "1. Start\n";
    cout << "2. Settings\n";
    cout << "0. Exit\n";
    cout << "Choose: ";
    cin >> choice;
} while (choice != 0);

cout << "Goodbye!";
```

Loop Differences

Difference between **FOR**, **WHILE**, and **DO-WHILE**

For Loop	While Loop	Do-While Loop
If the condition is not true at first time than control will never enter in a loop	If the condition is not true at first time than control will never enter in a loop	Even if the condition is not true at first time the control will enter in a loop
Initialization and updating is a part of syntax.	Initialization and updating is not a part of syntax.	Initialization and updating is not a part of syntax.

The **continue** statement skips the remaining code in the current loop iteration and moves to the **next iteration**.

Example:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        continue;  
    }  
    cout << i << " ";  
}
```

Output:

1 2 4 5

When i is 3, **continue** skips cout, so 3 is not printed.

continue skips the current iteration, but the loop continues running.

Break

The **break** statement immediately stops the loop and continues execution with the code after the loop.

Example:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        break;  
    }  
    cout << i << " ";  
}
```

Output:

1 2

When *i* becomes 3, **break** terminates the loop, so the loop does not continue to 4 and 5. **break** stops the entire loop, while **continue** skips only the current iteration.



KAZAKH-BRITISH
TECHNICAL
UNIVERSITY

Questions?

Lecture 2

z.bolatov@kbtu.kz



Zhiger Bolatov

Senior Lecturer
School of IT&I